Chapter 5

Retrieval: Finding Candidates at Scale

5.1 Introduction: The Retrieval Problem

In the previous chapter, we built the data infrastructure foundation that supports production recommender systems: feature stores, data pipelines, and quality monitoring. With that infrastructure in place, we are now ready to start building the models themselves. This chapter begins with the first and arguably most critical stage of the multi-stage pipeline: retrieval.

Retrieval is the process of narrowing a massive item catalog, often millions or even billions of items, down to a manageable set of few 1000s or 100s of candidates that downstream stages can evaluate in detail. This is the stage where scale and latency constraints are most severe. The subsequent stage's ranking model can afford to spend ~50 milliseconds scoring a few hundred candidates with rich features and cross-interactions. But you cannot run that model on every item in your catalog. Retrieval must identify the relevant few hundred items in single-digit milliseconds. Production ANN indexes such as FAISS and Milvus routinely achieve p95 latencies of 1–10 ms on billion-scale vector collections when the index fits in memory (Milvus, 2024), and end-to-end retrieval pipelines at companies like Spotify and DoorDash are typically budgeted at 10–50 ms p99 for the full candidate generation path (Shaped, 2024).

The fundamental tradeoff in retrieval is recall versus latency. A brute-force approach that scores every item against every user would achieve highest possible recall but is computationally infeasible at scale. The art of retrieval is finding representations and indexing strategies that achieve high recall, surfacing the items that a perfect ranker would place in its top results, while operating within strict computational budgets.

This chapter covers three families of retrieval techniques, progressing from simple to sophisticated: rule-based and heuristic retrieval (popularity, recency, business rules), sparse retrieval using text-based signals (BM25, SPLADE), and dense retrieval using learned embeddings (two-tower models). For dense retrieval, we will also cover Approximate Nearest Neighbor (ANN) indexing, which is the infrastructure that makes embedding-based retrieval feasible at scale. We conclude with practical guidance on building hybrid retrieval systems that combine multiple retrieval sources.



Figure 5.1: The retrieval stage in the multi-stage recommender pipeline. Retrieval takes the full item catalog (millions) and produces a candidate set (hundreds) that is passed to the ranking stage.

5.2.1 Popularity-Based Retrieval

The simplest retrieval strategy is to return the most popular items globally. Popularity can be measured by total interactions, recent interactions (trending), or purchase frequency. This approach requires no personalization and no model, just a sorted list of item IDs by engagement count.

Popularity-based retrieval is often used as:

- A fallback for cold-start users who have no interaction history.
- A component in blended retrieval alongside personalized sources.
- An A/B test control group to measure the incremental value of personalization.

```
class PopularityBaseline:
    def __init__(self, engagement_counts):
        self.sorted_items = sorted(
            engagement_counts.keys(),
            key=engagement_counts.get, reverse=True
        )

    def retrieve(self, history_items, top_k=10):
        return [item for item in self.sorted_items
                if item not in history_items][:top_k]
```

5.2.2 Recency-Based Retrieval

In domains with rapidly changing content (news feeds, social media, live commerce), recency is a powerful retrieval signal. Simply surfacing the most recently published or updated items can be surprisingly effective. Many production systems maintain a “new items” retrieval source that injects fresh content into the candidate set, ensuring that newly onboarded items get exposure before any engagement data accumulates.

5.2.3 Business Rule Retrieval

In e-commerce and retail media, business rules often generate candidates: items on promotion, items with high inventory that need to be moved, seasonal merchandise, or editorially curated collections. These are not ML-driven, but they represent legitimate

retrieval sources that feed into the same ranking pipeline. The key principle is that retrieval is a funnel, not a single model. Multiple retrieval sources can contribute candidates to a shared candidate pool, which is then scored by the ranker.

5.3 Sparse Retrieval: BM25 and Learned Sparse Models

A purely keyword/lexical approach can be used when the user provides an explicit query (such as in Information Retrieval or Search) or when the driver item metadata contains primarily rich text (e.g., product title) that we can use as the pseudo-query. Popular algorithms include BM25 for keyword search and SPLADE.

5.3.1 Dense vs. Sparse: Two Approaches to Representing Text

Before introducing SPLADE, it is worth stepping back to dense vs sparse representations. Both BM25 and SPLADE produce **sparse representations**, while two-tower models (Section 5.4) produce dense representations. Sparse representations encode an object (document/query/user/item) in a *very high-dimensional* space where most dimensions are zero, and only a small set of features are non-zero. In retrieval, those dimensions usually correspond to vocabulary terms (or expanded terms), e.g. a map like { "samsung": 3.2, "galaxy": 1.8, ... }.
Dense representations encode an object as a *low- to medium-dimensional* vector where every dimension has a value, e.g. a 768-d embedding like [0.12, -0.34, ...].

These two paradigms make fundamentally different trade-offs in how they encode text, and understanding the contrast clarifies why modern systems deploy both.

Consider encoding the product text “*Samsung Galaxy S24 Ultra Smartphone*”. A dense embedding produces a compact vector where every dimension has a value, for example [0.12, −0.34, 0.56, 0.01, −0.89, 0.23, …] with 768 dimensions. Each dimension represents a learned latent feature. These dimensions are not human-interpretable, you cannot point to dimension 247 and say “this one encodes brand identity.” Instead, the semantic information is distributed across all 768 values.

A sparse embedding produces a high-dimensional vector where more than 99% of dimensions are zero. Instead of storing the zeros, we store the non-zero entries as a dictionary keyed by vocabulary term: {“samsung”: 3.2, “galaxy”: 1.8, “s24”: 1.5, “ultra”: 2.9, “phone”: 1.4, “wireless”: 0.4, “mobile”: 0.6, …}. Each non-zero dimension corresponds to a specific vocabulary term, making the representation fully interpretable—you can read off exactly which terms the product “activates” and how strongly.

These two paradigms differ on nearly every axis, as summarized in Table 5.1 below.

Table 5.1: Key differences between dense and sparse text representations.

Property	Dense (e.g., Sentence-BERT)	Sparse (e.g., BM25, SPLADE)
Dimensionality	128–768	~30,000 or more depending on vocabulary size.
Sparsity	Every dimension has a value	>99% zeros
Index structure	Approximate Nearest Neighbor (ScaNN, HNSW, IVF)	Inverted index
Scoring	Dot product over full vector	Dot product over non-zero overlap
Strength	Semantic similarity; paraphrase; synonyms	Exact lexical matching; brand, model no., SKU ID
Weakness	Misses exact keyword matches	BM25 misses synonyms; SPLADE partly fixes this
Interpretability	Black box because of latent dimensions	Semi/Transparent where vocabulary terms can be inspected
Zero-shot generalization	Moderate; benefits from in-domain fine-tuning	BM25 – not applicable since it's a formula. SPLADE often performs poorly on out-of-domain.

In text-heavy domains like search, neither paradigm dominates, both are required to get good results. Dense retrieval shines when the user's vocabulary differs from the document's vocabulary ("running shoes" vs. "sneakers"); sparse retrieval shines on exact identifiers (SKUs, model numbers, brand names) and on out-of-domain corpora where fine-tuning data is scarce. This is why modern search systems typically deploy both and fuse their results, a pattern covered in Section 5.9 and explored empirically in the Appendix on Hybrid Search with Vector Databases.

5.3.2 The Inverted Index: The Foundation of Lexical Search

Even though ML Engineers rarely are tasked with building or maintaining a search engine, it helps to understand the data structure that makes sparse retrieval tractable at scale: the inverted index. Production text search engines are built on this foundation, and both BM25 and the learned-sparse models we discuss are typically served through the same indexing machinery.

Consider a naive approach to searching a product catalog: given a query like "samsung wireless phone," scan every product description and count how many query terms each one contains. With 50,000 products this is slow; with a million it is intractable. The bottleneck is the direction of the lookup.

Forward Index

"What terms are in this document?"

Document	Terms
Doc 1 (Samsung Galaxy S24)	samsung, galaxy, s24, wireless, phone
Doc 2 (Sony WH-1000XM5)	sony, noise, cancelling, wireless, headphones
Doc 3 (LG C3 OLED TV)	lg, oled, smart, tv, wireless

Inverted Index

"Which documents contain this term?"

Term	Posting List (doc IDs)
samsung	Doc 1
wireless	Doc 1, Doc 2, Doc 3
tv	Doc 3
galaxy	Doc 1
noise	Doc 2

Figure 5.2: Forward index versus inverted index for a small product catalog. The forward index answers "what terms are in this document?" and represents naïve text storage where each row lists the terms contained in one product. The inverted index flips this relationship to answer "which documents contain this term?" Each row lists the documents in which one term appears (the term's posting list)

A forward index answers the question “what terms are in this document?” Where for each product, we store the list of terms it contains. An inverted index flips this around to answer “which documents contain this term?” Where for each term, we store the list of document IDs it appears in (called the term’s posting list). With an inverted index, the query “samsung wireless phone” becomes three fast lookups followed by a set intersection: look up the posting list for each term, then intersect them to find documents containing all three. No full catalog scan is needed.

Beyond boolean matching: weighted posting lists. Real inverted indexes store more than just document IDs in each posting list: they store a weight for each (term, document) pair that encodes how important that term is to that document. A term like “samsung” appearing in only 1 out of 50,000 products gets a high weight because it is highly discriminative: knowing a product contains it tells you a lot. A term like “wireless” appearing in 40,000 out of 50,000 products gets a very low weight because it barely distinguishes products from one another. The job of a scoring function like BM25 is to compute weights intelligently so that high-weight matches surface first at query time.

Now that we understand some conceptual foundations of retrieval, lets move onto the algorithms and models used.

5.3.3 BM25: The Enduring Baseline

BM25 (Best Matching 25) is a term-frequency–based scoring function that has been the standard text retrieval baseline for over two decades. BM25’s strengths are that it

- requires no training data (this is not an ML model),

- handles exact keyword matching well (which learned models sometimes miss),
- is computationally efficient via inverted indexes,
- and is highly interpretable

Lets see how the BM25 implementation looks like. For a query q and document d, the BM25 score is:

$$\text{score}(q,d) = \sum_{t \in q} \text{IDF}\,(t) \cdot \frac{\text{tf}(t,d)\,(k_1+1)}{\text{tf}(t,d) + k_1\left(1 - b + b\frac{|\,d\,|}{\text{avgdl}}\right)}$$

where tf(t,d) is the term frequency of term t in document d, IDF(t) is the inverse document frequency, |d| is the document length, *avgdl* is the average document length, and k_1 and b are tunable parameters (typical defaults: k_1 = 1.2, b = 0.75).

The formula can look intimidating, but there are three components, each encoding a specific intuition about relevance.

1. **Inverse Document Frequency** IDF(t) measures term rarity across the corpus. Rare terms are more informative than common ones.
    a. If “samsung” appears in 2 of 50,000 documents, it has a high IDF: knowing a document contains it is very informative.
    b. If “wireless” appears in 40,000 of 50,000 documents, its IDF is low because nearly every product is wireless, so the term barely discriminates between candidates.

2. **Term-frequency factor** $\frac{\text{tf}(t,d)\,(k_1+1)}{\text{tf}(t,d)+k_1}$ counts how often a term appears in a document but with diminishing returns.
    a. The first mention of “samsung” in a product title is highly informative; the third mention adds much less signal.
    b. As tf grows, the expression asymptotes toward k_1 + 1.
    c. The hyperparameter k_1 (typically 1.2–2.0) controls how quickly term frequency saturates: higher values delay saturation and let repeated terms contribute more.
3. **Document length normalization factor** $1 - b + b \cdot \frac{|d|}{\text{avgdl}}$ in the denominator penalizes longer documents.

a. Longer documents match more terms by chance simply because they contain more words, which would unfairly boost their scores (and also makes the ecommerce catalog gameable if you are running a marketplace)
b. The ratio $\frac{|d|}{\text{avgdl}}$ captures how long this document is relative to the corpus average; the hyperparameter b (typically 0.75) controls how aggressively length is penalized. Setting b = 0 disables normalization entirely; b = 1 applies full normalization.

BM25's weights are purely statistical: computed from corpus term-frequency counts, with no relevance labels and no learning. It is parameter-efficient (no model training), interpretable (each term's contribution to the score is visible), and surprisingly strong as a baseline. Even now, production e-commerce **search** systems still run some variation of BM25 as a first-stage retriever or as one signal in a hybrid ranking pipeline.

Its primary weakness is the vocabulary mismatch problem: if the user searches for "sneakers" but the product title says "running shoes", BM25 will miss the match because the terms do not exactly match. BM25's strength as well as its limitation is that its a bag of words. BM25 treats each document as an unordered multiset of terms. It has no notion of word order, adjacency, or semantics, which leads to two specific failure modes beyond the vocabulary mismatch mentioned above.

The first failure is no phrase matching. Consider the query "wireless noise cancelling headphones" and the catalog containing two candidate products: (a) "Wireless Noise Cancelling Headphones — Sony WH-1000XM5" and (b) "Wireless keyboard with noise dampening, ships with headphone stand." BM25 will score these roughly equally because both contain the same query terms at similar frequencies. A human reader immediately sees that (a) is a near-exact match and (b) is barely related, but BM25 cannot see phrase structure.

The second failure is no semantic understanding. For the query "energy efficient large screen TV" against the product "65-inch 4K OLED display with eco mode," BM25 scores the product zero because no query terms appear in the product text having failed to not recognize that the 'eco' maybe related to being energy efficient. This is the vocabulary mismatch problem in its sharpest form, and it motivated two different successor technologies: dense retrieval, which solves semantics in a continuous embedding space (Section 5.4), and learned sparse retrieval like SPLADE, which solves semantics while preserving the inverted-index-friendly sparse representation (Section 5.3.4).

5.3.4 SPLADE: Learned Sparse Retrieval

SPLADE (SParse Lexical AnD Expansion) is a pre-trained model that bridges the gap between BM25 and dense retrieval by learning sparse, vocabulary-sized vectors that retain the efficiency of inverted indexes while addressing the vocabulary mismatch problem.

The architecture is as follows:

SPLADE builds on a pretrained masked language model (typically DistilBERT or BERT-base) trained on large-scale text corpora (English Wikipedia and the BookCorpus, totaling roughly 3.3 billion tokens) using the Masked Language Modeling (MLM) objective. During MLM pretraining, the model learns to predict masked tokens from context, which means its output vocabulary logits already encode rich semantic relationships between words. SPLADE repurposes this capability: rather than predicting a single masked token, it uses the MLM head's logits across the entire vocabulary to produce a sparse representation of the input text. The SPLADE-specific fine-tuning is then performed on information retrieval datasets (commonly the MS MARCO which is passage ranking dataset of ~8.8 million passages derived from Bing search queries logged in mid-2010s) using contrastive losses with hard negatives mined from BM25 results.

The neural architecture producing the vector is as follows: Input Text → Transformer (BERT/DistilBERT) → MLM Head (vocabulary logits) → SPLADE Pooling → Sparse Vector.

Concretely, for each token position in the input, the MLM head produces a logit vector over the full vocabulary (~30,000 dimensions for BERT's WordPiece tokenizer). SPLADE pooling then applies a log-saturation transform, log(1 + ReLU(logit)), to each logit and takes the element-wise maximum across all token positions. The result is a single sparse vector where each dimension corresponds to a vocabulary term and its value indicates how relevant that term is to the input. The log(1 + x) saturation prevents any single term from dominating the representation, while the ReLU ensures sparsity by zeroing out negative logits. The final vectors are typically very sparse, with only 100–300 non-zero entries out of ~30,000 dimensions. The formula is:

```
w_j = max_over_tokens[ log(1 + ReLU(mlm_logit_j)) ]
```

The key insight is that the MLM (Masked Language Model) head was pretrained to predict what words could fill a [MASK] position. SPLADE repurposes this capability: given "Samsung Galaxy S24," the MLM head activates related vocabulary terms like "phone," "mobile," "android," "5g", even though they do not appear in the input. This is how SPLADE solves the vocabulary mismatch problem that limits BM25.

SPLADE vectors are sparse (~100–300 non-zero entries out of ~30,000 vocabulary dimensions), which means they can be stored and queried using the same inverted index infrastructure as BM25. This is a significant practical advantage: you can replace or augment BM25 with SPLADE without changing your retrieval infrastructure.

```
from sentence_transformers import SparseEncoder
from sentence_transformers.sparse_encoder.models import (
    MLMTransformer, SpladePooling
)
from sentence_transformers.sparse_encoder.losses import (
    SpladeLoss, SparseMultipleNegativesRankingLoss
)

# Initialize sparse encoder from a pretrained MLM model
model = SparseEncoder(modules=[
    MLMTransformer("distilbert/distilbert-base-uncased"),
    SpladePooling(pooling_strategy="max"),
])

# Loss: contrastive ranking + FLOPS regularization for sparsity
inner_loss = SparseMultipleNegativesRankingLoss(model=model)
loss = SpladeLoss(
    model=model, loss=inner_loss,
    query_regularizer_weight=5e-5,
    document_regularizer_weight=3e-5,
)
```



Figure 5.3: Comparison of BM25 and SPLADE retrieval. BM25 matches only exact terms between query and document. SPLADE expands both query and document into related vocabulary terms, enabling semantic matching while maintaining sparse, inverted-index-compatible representations.

Same pipeline, different engine. An easy way to understand SPLADE is to place it side-by-side with BM25. Both pipelines produce sparse vectors in the same vocabulary

space, and both can be served from an inverted index. The only thing that changes is the weighting mechanism that populates the sparse vector, as summarized in Table 5.2.

Table 5.2: BM25 and SPLADE produce sparse vectors through the same pipeline shape. The critical difference is that BM25 outputs only terms that appear in the input, while SPLADE expands to related terms (synonyms, attributes, co-occurring concepts) that never appeared in the query or document.

Pipeline stage	**BM25**	**SPLADE**
Input	“wireless headphones”	“wireless headphones”
Tokenizer	WordPiece / whitespace	WordPiece (BERT)
Weighting engine	TF-IDF over corpus statistics (no training)	BERT + MLM head, log(1 + ReLU(logits)) (trained on query–passage pairs)
Output (sparse vector)	{10287: 1.8, 20164: 0.9} — only terms present in input	{10287: 2.1, 20164: 1.4, 18223: 0.6, 4891: 0.3} — input terms plus expansions (“bluetooth”, “earbuds”, “audio”)
Training requirement	None	~500K query–passage pairs with relevance labels
Index structure	Inverted index	Inverted index (same infrastructure)

Statistical vs. learned weights. This expansion capability is a direct consequence of where the weights come from. BM25’s weights come from corpus term-frequency statistics—they are computed after indexing, require no training data, and carry no relevance signal. “Wireless” gets a low weight under BM25 because it appears in many documents (low IDF)—a purely statistical property of the corpus.

SPLADE’s weights, in contrast, come from gradient descent on roughly 500,000 real query–passage pairs with relevance labels. They are optimized for retrieval quality and are contextual and search-aware. “Wireless” also gets a low weight under SPLADE, but for a different reason: the model learned that assigning it a high weight caused irrelevant products to score too highly during training, so it downweighted the term. Both approaches often agree on generic terms (“the,” “product,” “wireless” all end up with low weights under both), but they can disagree on domain-specific terms. In an electronics catalog, “flagship” may have moderate IDF under BM25 (the term is moderately common), yet SPLADE may learn to assign it a high weight because users who search for “flagship” strongly prefer premium products. SPLADE captures relevance signals that pure statistics cannot.

Publicly released SPLADE checkpoints are typically trained on MS MARCO: approximately 500,000 query–passage pairs derived from Bing web search queries, with binary relevance labels (0 or 1) and typically one relevant passage per query. MS MARCO is an excellent dataset for open-domain web retrieval, but it has meaningful gaps for e-commerce product search. It contains no product attributes (size, color, specs), no price-sensitivity signals ("cheap," "premium," "flagship" are not grounded in price), no brand preference patterns, no inventory or availability awareness, and the style of web queries ("how to fix a leaky faucet") differs substantially from product queries ("chrome bathroom faucet 8-inch centerset").

If you need to improve[1] the performance, fine-tune SPLADE on internal search logs if you have query-click data or editorial relevance labels. The Appendix on Hybrid Search with Vector Databases contains a detailed empirical comparison of three SPLADE variants on the Amazon ESCI dataset, including a domain-fine-tuned checkpoint and the pretrained MS MARCO baseline.

5.3.5 When to Use Sparse vs. Dense Retrieval

Having covered the sparse retrieval side (BM25, SPLADE) and prepared the ground for dense retrieval (Section 5.4), we can now summarize when each paradigm is preferred. The choice depends on query type, infrastructure constraints, training data availability, and how much personalization the system needs. Table 5.3 below captures the main trade-offs.

Criterion	Sparse (BM25/SPLADE)	Dense (Two-Tower)
Query type	Explicit text queries (search)	Implicit signals (user history, context)
Exact keyword matching	Excellent	Can miss exact matches
Semantic matching	BM25: poor; SPLADE: good	Excellent
Cold-start items	Works if item has text metadata	Works if item has text metadata. But typically requires user sequences.
Infrastructure	Inverted index (Elasticsearch)	Vector database (FAISS, Milvus)
Training data required	BM25: none; SPLADE: query-doc pairs	User-item interaction logs
Personalization	Limited (query-dependent only)	Strong (user history encoded)

[1] Although SPLADE demonstrated strong zero-shot generalization on public benchmarks like BEIR (where it was performing competitively across 18 diverse retrieval tasks without task-specific fine-tuning), treat this model with caution if your domain differs substantially from web search. In the author's experience, SPLADE performs poorly for ecommerce.

5.4 Dense Retrieval: The Two-Tower Model

Dense retrieval using two-tower models is the workhorse of personalized candidate generation in modern recommender systems (in contrast to Search/IR domains, where a hybrid approach is preferred with heavier emphasis on the lexical search part). This section covers the architecture in depth.

5.4.1 Architecture Overview

The two-tower model (also called a *dual encoder* or *twin-tower* model) learns separate representations for users (or queries) and items, then scores their compatibility via a simple similarity function, typically the cosine distance or dot product, in a shared embedding space.

The model consists of two neural network "towers." The user tower (or query tower) processes user features (ID, interaction history, context) to produce a user embedding vector. The item tower processes item features (ID, category, text, images) to produce an item embedding vector. The predicted affinity is the dot product (or cosine similarity) of these two vectors.



Figure 5.4: Simplified two-tower architecture. The user tower takes user features (e.g., browsing history) and produces a user embedding u. The item tower takes item features (ID, category, description, image) and produces an item embedding v. The predicted affinity is the dot product u · v

For readers building conversational AI systems where vector similarity search is the first step in a RAG pipeline, the two-tower model can be thought of as the retrieval component fine-tuned to your user interaction data. With RAG, the incoming query is typically vectorized through an off-the-shelf embeddings model, whereas in RecSys, the

towers are trained on the historical interaction data and the “query” side encodes much more than just text, it encodes the user’s full behavioral context.

5.4.2 Training Objectives and Loss Functions

The two towers are trained jointly on historical user-item interaction data. Training data consists of tuples (user, item, label) where label is 1 for a positive interaction (click, purchase, watch) and 0 for non-interactions. The objective is to maximize the dot product $u \cdot v$ for positive pairs while minimizing it for negative pairs. Common loss functions include:

Pointwise losses: Treat each pair as binary classification using cross-entropy on the dot product score.

Pairwise ranking losses: Compare a positive item to a sampled negative item for the same user (e.g., BPR loss, hinge loss).

Contrastive losses (InfoNCE): Treat each user’s interacted item as the correct class among a set of negatives. With in-batch negative sampling, a batch of N user-item pairs yields N positive pairs and N(N−1) negative pairs.

InfoNCE (Noise-Contrastive Estimation), introduced by Oord et al. (2018) in the context of representation learning, is the loss function behind many industrial retrieval/ranking models. The core idea is to frame retrieval as a classification problem: given a user embedding q and a set of item embeddings $\{k_0, k_1, \ldots, k_n\}$ where k_0 is the positive (interacted) item and the rest are negatives, the loss is: $L = -\log(\exp(q \cdot k_0 / \tau) / \sum_i \exp(q \cdot k_i / \tau))$, where τ is a temperature hyperparameter that controls the sharpness of the distribution. Lower temperatures make the model more discriminative but can make optimization harder. This is equivalent to a softmax cross-entropy loss where the positive pair should have the highest dot-product similarity among all candidates.

What makes InfoNCE particularly effective for two-tower models is its natural pairing with in-batch negative sampling. In a training batch of N user-item pairs, each user’s positive item serves as a negative for every other user in the batch, yielding $N^2 - N$ negative pairs “for free” without any additional sampling. This means larger batch sizes directly improve training signal quality by providing more and harder negatives. Google’s retrieval systems, YouTube’s two-tower model, and Meta’s embedding-based retrieval all use variants of this contrastive formulation. The temperature parameter τ is typically tuned in the range 0.05–0.1 and has a significant impact on learned representation quality.

```
# In-batch contrastive loss (InfoNCE)
# user_emb: (batch_size, embedding_dim)
# item_emb: (batch_size, embedding_dim)
```

```
logits = torch.matmul(user_emb, item_emb.T) / temperature
labels = torch.arange(batch_size, device=logits.device)
loss = F.cross_entropy(logits, labels)
```

This loss is computationally elegant: each user's ground-truth item serves as a negative for every other user in the batch, providing N−1 negatives per sample without any additional sampling overhead.

5.4.3 Negative Sampling Strategies

It is common in the industry to introduce additional negative samples to enrich the dataset during training. The choice of how many/which additional negative samples to choose in training profoundly affects the trained model quality. Key strategies include:

In-batch negatives: This is a common approach where we specify to the training algorithm to use other items in the mini-batch as negatives. This is computationally 'free' and provides diverse negatives for the model to learn differences.

Popularity-based negatives: Popular items are plausible candidates that the user chose not to interact with, making them informative negatives. The key idea is to sample negatives from a random set, proportional to item popularity. However, over-reliance on popularity negatives can cause the model to merely distinguish popular from unpopular items rather than truly personalizing.

Hard negatives: 'Hard' in this context are those items that the current model scores highly but the user did not interact with. Introducing these negatives in the model training, forces the model to make finer distinctions. Investment in finding hard negatives is needed when you want the retrieval/ranking model to be super precise in the topK positions. Facebook's search team found that mixing easy and hard negatives in a 100:1 ratio worked best: just using easy hard negatives caused overfitting, while a small fraction of hard negatives significantly improved recall (+8% in people search).

Author's Note: *Negative sampling is one of the highest-leverage levers in two-tower model training. If your model is underperforming, before reaching for a more complex architecture, try improving your negative sampling strategy. A two-tower model with well-chosen hard negatives will often outperform a more sophisticated architecture with naive random negatives or no negative sampling. This is also an area where offline usage of LLMs to sample negatives can assist.*

5.4.4 The User Tower: Encoding Interaction History

The user tower must compress a variable-length interaction history into a fixed-size embedding vector. Common encoding strategies include:

Mean Pooling: Average the embeddings of all items in the user's history. Simple and fast, but treats all items equally regardless of recency or relevance.

Last-Item Encoding: Use only the embedding of the most recently interacted item. Captures current intent well for session-based recommendations.

Attention Pooling: Learn a scalar weight for each history item and compute a weighted sum. Each item's weight is computed independently based on its embedding. This is not the same as Transformer self-attention; it is simpler "additive attention" used for sequence pooling.

```
class AttentionPooling(nn.Module):
    def __init__(self, embedding_dim, hidden_dim):
        super().__init__()
        self.attention = nn.Sequential(
            nn.Linear(embedding_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, 1)
        )

    def forward(self, x, mask):
        scores = self.attention(x).squeeze(-1)
        scores = scores.masked_fill(~mask, float('-inf'))
        weights = F.softmax(scores, dim=-1)
        return torch.bmm(weights.unsqueeze(1), x).squeeze(1)
```

Transformer Encoder: Full self-attention over the history sequence, using a [CLS] token to aggregate. Most expressive but also most parameter-heavy.

Simple baselines are often sufficient. Complexity doesn't guarantee better performance. In the author's experience, last-item aggregation works well in e-commerce. Supervised models with more complex ingestion patterns for user signals tend to outperform heuristics when some combination of this exists in the training data distribution:

a) user histories are longer (providing more signal to aggregate),

b) there is substantially more training data,

c) and/or the task involves complex intents that simple heuristics cannot capture.

5.4.5 The Item Tower and Serving Architecture

The primary appeal of the two-tower architecture is its scalability at inference time. Since the item embedding v does not depend on the user, all item embeddings can be precomputed offline and stored in an ANN index. At query time, the system only needs to compute the user embedding u (one forward pass through the user tower), then

perform a nearest-neighbor search in the precomputed item index to find the top-K items with the largest dot product u · v.

This decoupling is what makes the two-tower model the de facto standard for candidate generation at scale. The serving architecture typically works as follows:

1. A user request arrives at the recommendation service.
2. The user-tower service computes the user embedding using the user's features and recent history.
3. The user embedding is sent to the ANN index service, which returns the top-K item IDs and their retrieval scores.
4. The candidate IDs are passed to the ranking service for detailed scoring.

Embedding freshness is a practical concern. Item embeddings should be recomputed when item features change or when the model is retrained. User embeddings are typically computed on-the-fly at request time, but some systems precompute and cache user embeddings as well, updating them every few hours. Snapchat, for example, precomputes user embeddings and story embeddings on different cadences, with story embeddings refreshed more frequently given rapid content churn.

5.4.6 Handling Cold-Start Items and Long-Tail

Real-world item catalogs are heavily skewed: a small fraction of items accounts for the majority of interactions (the "head"), while a long tail of items has very few interactions. This creates challenges for embedding-based retrieval:

Frequency-based vocabulary curation: YouTube's system (Covington et al., 2016) truncated very large ID spaces to the top-N most frequent video IDs, mapping all others to an out-of-vocabulary embedding. Super-rare items share a single catch-all representation.

Content features for cold-start: When an item is new or very rare, its ID embedding has had little opportunity to learn. Incorporating item metadata (text descriptions, images, category attributes) into the item tower allows the model to compute a reasonable embedding for new items based on their content. Amazon Prime Video researchers found that using a two-tower model with item metadata (genre, description text, cover image) significantly improved recommendations for cold-start titles.

Note: We will revisit the topic of item embeddings in in Chapter 10, where we discuss learned representations that can serve multiple recommendation surfaces simultaneously. The embeddings discussed in this chapter are specific to the retrieval task; Chapter 10 generalizes this concept to embeddings that are shared across retrieval, ranking, notifications, and other use cases. Similarly, Chapter 11 covers user

embeddings as reusable representations. The cold-start and long-tail strategies mentioned here are expanded upon in those chapters.

5.5 Approximate Nearest Neighbor (ANN) Search

With millions of items, computing the exact nearest neighbors to a query vector is prohibitively expensive. ANN algorithms trade a small amount of recall for dramatic speedups, enabling retrieval in milliseconds.

5.5.1 Why ANN Is Necessary

Brute-force nearest neighbor search over N items with D-dimensional embeddings costs O(N × D) per query. For a catalog of 100 million items with 128-dimensional embeddings, that is 12.8 billion floating-point operations per query which is far too slow for real-time serving. ANN algorithms reduce this to roughly O(D × log N) or better, at the cost of occasionally missing a true nearest neighbor.

5.5.2 Major ANN Algorithms

Algorithm	How It Works	Strengths	Weaknesses
IVF	Clusters items into Voronoi cells; searches only nearby clusters at query time	Small memory footprint; supports filtered queries; easy to update	Requires good clustering; recall drops if nprobe is too low
HNSW	Builds a multi-layer graph connecting similar items; query traverses graph greedily	Excellent recall-speed tradeoff; no training required	Large memory footprint (stores graph edges); difficult to update
PQ	Compresses vectors by splitting into subvectors and quantizing each independently	Dramatic memory reduction (10–100×); can combine with IVF	Lossy compression reduces recall; requires codebook training

The choice[2] of ANN algorithm should be guided by your operational constraints, not just benchmark numbers. Airbnb found that IVF was more suitable than HNSW for their use case because IVF's smaller memory footprint and its natural support for filtered queries (treating cluster IDs as filters) aligned better with their search system's requirements. They also discovered that using Euclidean distance instead of dot product yielded more

[2] *If you are implementing retrieval, it's a reasonable choice to go by the default search/vector db tool's support for the initial MVP launch. Then, in later iterations come back and understand the impact of different ANN algorithms.*

balanced clusters, improving retrieval performance. Always prototype with your actual data and query patterns.

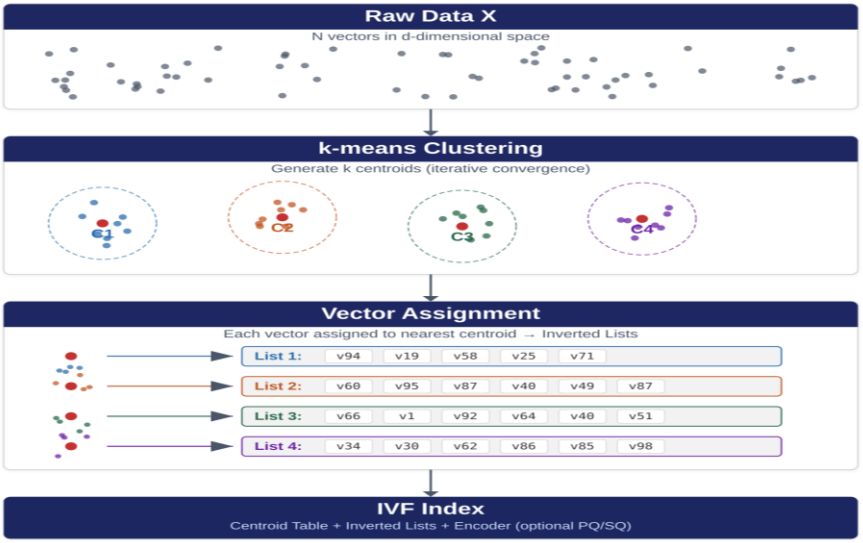


Figure 5.5: Building an IVF index. Raw item embeddings are partitioned into clusters using k-means, where each cluster is defined by its centroid. Every embedding is then assigned to its nearest centroid, forming inverted lists — one per cluster — that map each centroid to the set of vectors in its partition. The resulting IVF index stores these inverted lists for efficient lookup at query time.

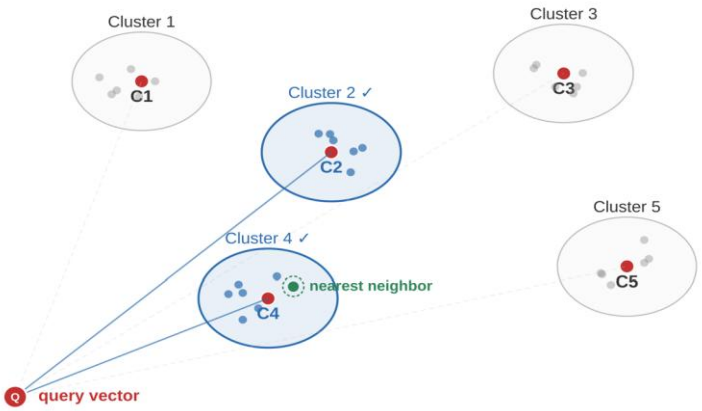


Figure 5.6: Searching an IVF index. Given a query vector, the index computes distances to all cluster centroids and selects the nprobe nearest clusters (here, nprobe = 2). Only the vectors within those selected clusters are compared to the query, yielding the approximate nearest neighbor. This reduces time complexity from O(N) to O(nprobe × N/nlist), trading a small recall loss for a large speedup.

5.6 Why Retrieval and Ranking Are Separate Stages

Before diving into retrieval techniques, it is worth understanding why retrieval and ranking are typically implemented as separate services in production. The separation exists for fundamental engineering reasons.

Compute profiles differ. The retrieval stage performs one forward pass through a user encoder (or a simple query formulation), followed by an approximate nearest neighbor lookup. This is memory-bandwidth intensive but computationally light. The ranking stage runs a full model evaluation per candidate item, using rich cross-features between the user and item. This is compute-intensive. These different resource profiles mean that retrieval and ranking services scale differently and benefit from independent autoscaling.

Update cadences differ. Item embeddings for retrieval are either bulk-refreshed on a periodic schedule or on demand. Rebuilding the ANN index on the other hand is a heavyweight operation. Ranking model weights, by contrast, can be retrained and deployed daily or even more frequently, and A/B tests on the ranker can be swapped in minutes. If retrieval and ranking were bundled in a single service, experimenting with one would require redeploying the other.

Failure isolation matters. If the ANN index service goes down, the system can fall back to a cached or rule-based candidate list. If the ranking service is unavailable, the system can return candidates in retrieval score order. This graceful degradation is only possible if the two stages are decoupled.

Organizational alignment. In most organizations, retrieval infrastructure is owned by a platform or search infrastructure team, while ranking models are owned by an ML team. Service boundaries that align with team boundaries reduce coordination overhead.

Dimension	Retrieval (Two-Tower + ANN)	Ranking (Deep Models, GBDT)
Compute per request	1 user-tower forward pass + ANN lookup	1 model evaluation per candidate (50–500×)
Memory profile	Large item embedding index (GBs)	Small model, many real-time features
Update cadence	Hourly/daily index rebuilds	Daily model retraining, rapid A/B swaps
Failure fallback	Cached/rule-based list	Return retrieval-order results
Typical team owner	Search/ANN infrastructure	ML ranking team

Separation of retrieval and ranking into distinct services is the norm at companies like YouTube, Uber Eats, Pinterest, LinkedIn, and Meta. YouTube presents the two components as distinct neural networks in its architecture, each tuned and A/B tested independently. Uber Eats explicitly names two production services: FPR (First-Page Retrieval) and SPR (Second-Page Ranking). That said, there are exceptions. TikTok's

"Monolith" system deliberately co-locates retrieval and ranking for ultra-low latency. Early-stage startups with modest traffic may also benefit from a simpler, co-located architecture.

5.6.1 Practical Deployment Patterns

In production, the separation of retrieval and ranking manifests in several common deployment patterns:

Side-by-side microservices (most common): A user-tower service returns the user embedding vector. A vector database service (FAISS, Milvus, Pinecone, or a bespoke ANN) returns the top-K item IDs and retrieval scores. A separate ranking service fetches per-item features (price, freshness, business rules) from the feature store and scores with a deep model or GBDT, returning the final list. This is the standard pattern at YouTube, Uber Eats, Pinterest, and LinkedIn.

Gateway orchestrator: A thin "rec-gateway" process fans out requests to both the retrieval and ranking services in parallel and stitches the response together. This keeps the external API surface stable while each tier iterates independently. The gateway also manages latency budgets, timeouts, and fallback logic.

Sidecar ranker for graceful degradation: Retrieval pods embed a lightweight heuristic ranker (for example, a linear combination of retrieval score and item freshness) that is used when the full ranking service is unavailable or exceeds its latency budget. The full ranker still lives in a separate fleet and is called when conditions allow. This pattern ensures the system can always serve a reasonable result even during partial outages.

5.6.2 When Co-Location Makes Sense

While the default recommendation is to keep retrieval and ranking separate, there are legitimate scenarios where co-locating them is warranted:

- **Ultra-low latency requirements:** When the p99 latency budget is under 2 milliseconds and the candidate set is very small (≤20 items), the network hop between services can become the dominant cost. Mobile keyboard suggestion engines and edge device recommenders fall into this category.
- **Early MVP with a single-engineer team:** For startup prototypes with modest traffic, the operational simplicity of a single service often outweighs the architectural benefits of separation. You can always split later when the need arises.
- **Unified real-time training loops:** TikTok's Monolith system deliberately co-locates the embedding table, ANN lookup, and ranker in one large worker set to

enable a tight real-time training loop. They compensate with extensive sharding and replica isolation, but this design is the exception, not the rule.

Even in co-located systems, experienced teams keep the interfaces between retrieval and ranking logically separated, for example, defining the ranker as a library with a clean API that could later be extracted behind gRPC if the need arises.

Start separate unless you have a very tight latency budget and a tiny candidate set. Create clear, versioned APIs between retrieval and ranking. This buys you independently tunable SLAs (retrieval QPS spikes do not throttle ranking experiments), faster iteration (ranking engineers can push new models and A/B tests without re-indexing the ANN), and graceful degradation paths (if the ranking fleet experiences a brownout, you can still serve a retrieval-only list). If you later prove that the extra network hop is your top bottleneck, and you cannot fix it with gRPC batching or co-location on the same node, then explore co-hosting. But most real-world systems hit resource-isolation and rollout-cadence issues long before network hops becomes the limiting factor.

5.3 Rule-Based and Heuristic Retrieval

The simplest retrieval methods do not involve any ML at all. They are important to understand for two reasons: they serve as baselines against which learned methods must demonstrate improvement, and they remain part of production systems as complementary retrieval sources even when ML-based retrieval is deployed.

5.7 Limitations of the Basic Two-Tower Model

Understanding the limitations of the basic two-tower architecture is important because it motivates both the ranking models in Chapters 6–7 and several advanced retrieval variants.

5.7.1 Limited Feature Interaction

By design, the user and item towers do not interact until the final dot product. The model cannot directly model cross-features between specific user attributes and item attributes. For example, a single-tower model could learn that “user is a gamer AND item is a gaming laptop” is a special combination, whereas the two-tower model must compress this into separate user and item vectors. This is why two-tower models typically underperform full cross-interaction models in ranking accuracy, and why the ranking stage (covered in Chapters 6–7) uses models that can exploit these cross-features.

5.7.2 Compressed Representations (Bottleneck)

The two-tower model forces all of a user's preference information into a single embedding vector (e.g., 128 dimensions). If a user has diverse interests (both sports and classical music), a single vector tends to blend or average these out, producing generic recommendations. This single-vector assumption is fundamentally at odds with the observation that user interests frequently span multiple, potentially unrelated topics.

5.7.3 Popularity Bias

Popular items appear in many users' positive interactions, so the model may embed them in a way that gives them high dot products with many users. This can overwhelm personalization, making top-K retrieved items too generic. Some implementations add an explicit bias term for items (analogous to matrix factorization with bias) to decouple global popularity from the learned embedding.

5.7.4 The Recall-Precision Tradeoff

Because of these limitations, the two-tower retrieval model is usually not the sole recommender. It excels at optimizing for the recall (finding a reasonably good set of candidates from a huge catalog) but not at precision (ordering those candidates perfectly). This is the accepted division of labor in multi-stage systems: retrieval optimizes for recall, ranking optimizes for precision.

5.8 Advanced Two-Tower Variants

Several directions have been explored to address the limitations of the basic two-tower model while retaining its serving efficiency. We briefly survey the most impactful ones; practitioners should evaluate these as potential improvements once a solid basic two-tower baseline is established.

5.8.1 Multi-Interest Models

Rather than representing a user as a single embedding, multi-interest models produce K separate interest vectors per user. At retrieval time, each interest vector queries the ANN index independently, and the results are merged. Alibaba's MIND model (Li et al., 2019), deployed on the Taobao mobile app, used a dynamic routing mechanism (inspired by capsule networks) to generate multiple user interest vectors. Online A/B tests showed significant improvements in both precision and recall.

5.8.2 Multi-Task Retrieval

Modern platforms optimize for multiple engagement signals (clicks, watch time, likes, shares, purchases). Rather than training separate retrieval models for each signal,

multi-task learning trains one two-tower model with multiple output heads. Snapchat's Spotlight two-tower model combined several positive interactions (long views, favorites, shares) into one composite training label, helping the model learn more robust user embeddings that account for various ways a user shows interest.

5.8.3 Knowledge Distillation

Rather than changing the two-tower architecture, knowledge distillation trains the two-tower model to mimic the predictions of a more powerful (but slower) teacher model, such as a cross-encoder. The student two-tower model learns richer representations while maintaining its serving efficiency. Hofstätter et al. (2020) demonstrated in their "Improving Efficient Neural Ranking Models with Cross-Architecture Knowledge Distillation" that a dual-encoder student trained with cross-encoder teacher scores can recover up to 95% of the cross-encoder's ranking quality while maintaining sub-millisecond inference. More recently, ERNIE-Search (Lu et al., 2022) introduced on-the-fly distillation during dual-encoder training, nearly closing the accuracy gap between two-tower and cross-encoder models on standard retrieval benchmarks.

5.8.4 Graph-Augmented Embeddings

When item-item relationship graphs are available (co-view networks, social graphs, product knowledge graphs), the item tower can be enhanced with a Graph Neural Network (GNN) to incorporate neighborhood information. Pinterest's PinSage model (Ying et al., 2018) used graph convolutions to aggregate information from neighboring nodes in a scalable way, reporting a 150% relative improvement in recall and 60% improvement in MRR over prior content-based approaches.

5.9 Building a Retrieval System with diverse candidates

Production retrieval systems rarely rely on a single retrieval source. Instead, they combine multiple retrieval methods, each contributing candidates to a shared pool that the ranker scores.

5.9.1 Multiple Retrieval Sources

A typical production retrieval system might include:

- **Personalized dense retrieval:** Two-tower model based on user history.
- **Search-based sparse retrieval:** BM25 or SPLADE for query-dependent results or for using prior session search queries as the input.
- **Similar-item retrieval:** Item-to-item embeddings for "more like this" use cases.
- **Popularity/trending retrieval:** For cold-start users or diversity injection.

- **Business rule retrieval:** Promotional items, editorially curated content.

Each source produces its own ranked list of candidates. The union of these lists (after de-duping) forms the candidate set for ranking. In practice, each source has a configurable quota (e.g., “retrieve 200 candidates from the two-tower model, 100 from BM25, 50 from trending”), and these quotas are tuned based on A/B test results.

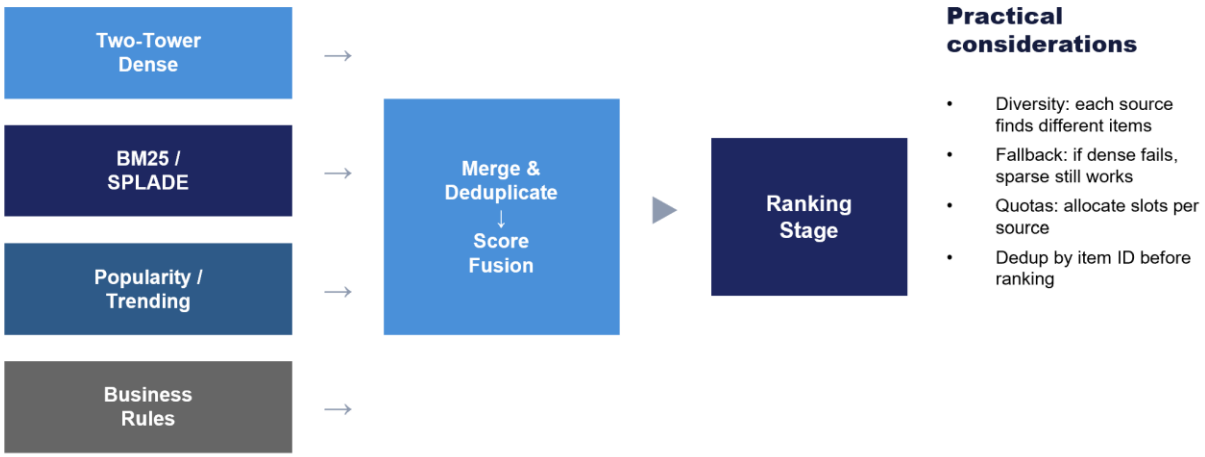


Figure 5.7: A hybrid retrieval system combining multiple retrieval sources. Each source contributes candidates to a shared candidate pool. The union of candidates is deduplicated and passed to the ranking stage. The quota for each source is a tunable parameter that balances personalization, relevance, diversity, and business objectives.

5.9.2 Practical Considerations for Hybrid Retrieval

Deduplication: The same item may be returned by multiple retrieval sources. Deduplication ensures each item appears only once in the candidate set.

Score normalization: Different retrieval sources may produce scores on different scales (cosine similarity, BM25 scores, popularity counts). If retrieval scores are passed as features to the ML-based ranker, they may need to be normalized to a common scale.

Latency budget allocation: Sources are typically queried in parallel, but the total latency is bounded by the slowest source. Monitor per-source latency and consider timeout policies.

5.10 Evaluation Metrics for Retrieval

The primary goal is to optimize for the recall metric: did the retrieval stage surface the items that a perfect ranker would have placed in the final results?

Metric	Definition	Use Case

Hit Rate@K	Fraction of queries where the ground-truth item appears in the top-K	Single ground-truth item (next-item prediction)
Recall@K	Fraction of all relevant items that appear in the top-K	Multiple relevant items per query
MRR@K	Mean Reciprocal Rank of the first relevant item in top-K	When position within K matters
nDCG@K	Normalized Discounted Cumulative Gain	Graded relevance (E/S/C/I labels)

Author's Note: *A practical tip on evaluation: always report retrieval metrics alongside several baselines, including at a minimum random baseline and a popularity baseline. Without these anchors, it is impossible to interpret whether a Hit Rate@10 of 15% is good or bad. In some use cases, due to quirks in the way in which relevancy is defined, metrics often saturate close to 1 and in such cases, the relative lift against the baseline 'models' serves as a better gauge for launch decisions.*

Time-based evaluation splits are best practice for retrieval evaluation: train on older interactions and evaluate on newer ones. This simulates realistic deployment where the model must predict future behavior from historical data. This also means that point-in-time user sequence history must be available for the hold out set.

5.11 Summary

This chapter covered the retrieval stage of the multi-stage recommender pipeline—the critical first step that narrows millions of items to hundreds of candidates. The key takeaways are:

5. Retrieval and ranking are typically separate services in production, with different compute profiles, update cadences, and failure modes.
6. Rule-based retrieval methods (popularity, recency, business rules) remain important as baselines and complementary retrieval sources.
7. Sparse retrieval (BM25, SPLADE) excels for search-like use cases with explicit text queries. SPLADE bridges the vocabulary gap through learned term expansion.
8. Dense retrieval via two-tower models is the workhorse of personalized candidate generation. The key innovation is decoupling user and item representations so that item embeddings can be precomputed and indexed for millisecond-latency retrieval.

9. ANN indexing (IVF, HNSW, PQ) makes embedding-based retrieval feasible at scale, trading small recall losses for dramatic speedups.
10. Simple baselines (last-item heuristic, mean pooling) can be surprisingly competitive. More parameters do not automatically mean better retrieval.
11. Production systems use hybrid retrieval, combining multiple sources to balance personalization, relevance, diversity, and business objectives.

In Chapter 6, we will cover the ranking stage, where the candidate set produced by retrieval is scored and ordered using models that can exploit rich cross-features between users and items.

References

[1] Robertson, S. & Zaragoza, H. (2009). “The Probabilistic Relevance Framework: BM25 and Beyond.” Foundations and Trends in Information Retrieval.

[2] Formal, T. et al. (2021). “SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking.” SIGIR 2021.

[3] Formal, T. et al. (2022). “From Distillation to Hard Negative Sampling: Making Sparse Neural IR Models More Effective.” SIGIR 2022.

[4] Covington, P., Adams, J., & Sargin, E. (2016). “Deep Neural Networks for YouTube Recommendations.” RecSys ’16.

[5] Huang, J.-T. et al. (2020). “Embedding-Based Retrieval in Facebook Search.” KDD ’20.

[6] Ying, R. et al. (2018). “Graph Convolutional Neural Networks for Web-Scale Recommender Systems.” KDD ’18. (PinSage)

[7] Li, C. et al. (2019). “Multi-Interest Network with Dynamic Routing for Recommendation at Tmall.” CIKM ’19. (MIND)

[8] Malkov, Y. & Yashunin, D. (2018). “Efficient and Robust Approximate Nearest Neighbor Using Hierarchical Navigable Small World Graphs.” IEEE TPAMI.

[9] Johnson, J., Douze, M., & Jégou, H. (2019). “Billion-Scale Similarity Search with GPUs.” IEEE Transactions on Big Data. (FAISS)

[10] Guo, R. et al. (2020). “Accelerating Large-Scale Inference with Anisotropic Vector Quantization.” ICML ’20. (ScaNN)

[11] Li, Z. et al. (2022). “IntTower: The Next Generation of Two-Tower Model for Pre-Ranking System.” CIKM ’22.

[12] Yu, X. et al. (2021). “Dual Augmented Two-Tower Model for Online Large-Scale Recommendation.” DLP-KDD ’21.

[13] Zhan, J. et al. (2021). “Optimizing Dense Retrieval Model Training with Hard Negatives.” SIGIR ’21.

[14] Snap Inc. (2023). “Embedding-Based Retrieval at Snap.” Snap Engineering Blog.

[15] Airbnb (2023). “Embedding-Based Retrieval for Airbnb Search.” Airbnb Engineering Blog.

[16] Oord, A. van den, Li, Y., & Vinyals, O. (2018). “Representation Learning with Contrastive Predictive Coding.” arXiv:1807.03748.

[17] Hofstätter, S. et al. (2020). “Improving Efficient Neural Ranking Models with Cross-Architecture Knowledge Distillation.” arXiv:2010.02666.

[18] Lu, W. et al. (2022). “ERNIE-Search: Bridging Cross-Encoder with Dual-Encoder via Self On-the-fly Distillation for Dense Passage Retrieval.” arXiv:2205.09153.

[19] Milvus (2024). “Vector Database Benchmarks.” Zilliz/Milvus Documentation. https://milvus.io/docs/benchmark.md

[20] Shaped (2024). “The Anatomy of Modern Ranking Architectures.” Shaped Engineering Blog.

ANN Libraries and Vector Databases

Tool	Type	Key Features
FAISS (Meta)	Library	GPU support, comprehensive algorithm suite, production-proven
ScaNN (Google)	Library	Optimized for dot-product search, quantization-aware
Annoy (Spotify)	Library	Memory-mapped indexes, simple API, good for read-heavy workloads
Milvus	Database	Distributed, supports hybrid search (vector + scalar filtering)
Pinecone	Managed service	Fully managed, real-time updates, metadata filtering
Qdrant	Database	Sparse + dense vector support, rich filtering, open-source